



RestAPI CI/CD

Com:

Golang
Docker
GitHub Actions
AWS

Gabriel Guerreiro Rocha Magalhães
Isabelle Bernardina da Silva Lopes
João Pedro Vasconcelos Rocha
João Victor Vieira Marcondes

Sumário

1. Conheça as tecnologias.....	1
1.1. Golang.....	1
1.1.1. Por que utilizar Go?.....	1
1.1.2. Quando utilizar Go?.....	1
1.2. RestAPI.....	2
1.3. Git.....	2
1.4. GitHub.....	2
1.4.1. Actions.....	3
1.4.2. Runners.....	3
1.5. Docker.....	4
1.5.1. DockerHub.....	4
1.6. AWS.....	4
1.6.1. EC2.....	5
1.7. CI/CD.....	5
2. O Projeto.....	5
3. Passo a passo.....	6
3.1. Pré-requisitos.....	6
3.2. Aplicação Go.....	6
3.3. Dockerização.....	7
3.4. Criação do Repositório no GitHub.....	9
3.5. Segredo no GitHub.....	11
3.6. Criar Repositório no DockerHub.....	12
3.7. Preparar VM.....	13
3.8. Self-Hosted Runner.....	20
3.9. Action CI.....	23
3.10. Action CD.....	25
4. Conclusão.....	28

1. Conheça as tecnologias

1.1. Golang

1.1.1. Porque utilizar Go?

Go, ou Golang, oferece várias vantagens para o desenvolvimento de REST APIs, especialmente devido ao seu desempenho, simplicidade e suporte nativo para recursos de rede. Como uma linguagem compilada, Go produz executáveis rápidos e eficientes, ideais para aplicações que exigem baixa latência e alta capacidade de resposta. Seu sistema de goroutines permite lidar com milhares de requisições simultâneas sem consumir muitos recursos, tornando-o excelente para cenários de alta concorrência, como APIs REST escaláveis. Além disso, a biblioteca padrão do Go inclui suporte nativo para servidores HTTP, o que elimina a necessidade de dependências externas para configurar e gerenciar uma API básica.

A filosofia por trás da linguagem é de não complicar, não desenvolver soluções para um problema já resolvido. Por isso, para muitas coisas temos apenas uma forma de fazer as coisas, o que leva a simplicidade e facilidade de ler e entender uma codebase já estabelecida. Mesmo quando o problema não é solucionado pela biblioteca padrão, a comunidade abraça a solução como se fosse, assim, temos bibliotecas bem estabelecidas, como por exemplo:

```
- gorilla/websockets para websockets;  
- pgx para postgres;  
- modernc/sqlite para sqlite;  
- jwt para autenticação e autorização;  
- cobra para CLIs;  
- viper para configuração (yaml, json, toml);  
- etc.
```

Outro ponto forte de Go é sua sintaxe simples e direta, que facilita o desenvolvimento e a manutenção de APIs. Os desenvolvedores podem criar aplicações robustas com menos código, reduzindo a probabilidade de erros e aumentando a produtividade da equipe. A comunidade ativa de Go e sua ampla coleção de pacotes também ajudam no desenvolvimento, oferecendo soluções para tarefas comuns como autenticação, validação de entrada e manipulação de dados. Por ser uma linguagem moderna e com foco em performance e confiabilidade, Go é uma escolha sólida para criar REST APIs que atendam tanto a startups quanto a sistemas empresariais em larga escala.

1.1.2. Onde Utilizar Go?

Go é amplamente utilizado em sistemas distribuídos e infraestrutura em nuvem devido à sua eficiência, concorrência nativa e simplicidade. Empresas como Google, Docker, Kubernetes, e HashiCorp adotaram Go para construir ferramentas e plataformas essenciais como Docker, Kubernetes e Terraform. A capacidade de lidar com milhares de conexões simultâneas torna Go ideal para servidores web, APIs REST e serviços de backend em geral. Além disso, sua rápida compilação e binários leves permitem implantações ágeis e escalabilidade em ambientes modernos, como microservices e arquiteturas serverless.

Outro domínio popular de Go é no desenvolvimento de ferramentas de linha de comando (CLIs) e automação de infraestrutura. Sua sintaxe clara e suporte para bibliotecas como Cobra facilitam a criação de utilitários poderosos e eficientes. Go também se destaca no processamento de dados em tempo real e tarefas de rede, como proxies reversos, balanceadores de carga e aplicações IoT. Graças à sua confiabilidade e desempenho, é uma escolha frequente em projetos que exigem alta disponibilidade e escalabilidade, como provedores de serviços em nuvem e plataformas SaaS.

1.2. RestAPI

Uma REST API (Application Programming Interface, ou Interface de Programação de Aplicações) é uma interface que permite a comunicação entre diferentes sistemas usando os princípios da arquitetura REST (Representational State Transfer). Essa arquitetura se baseia em protocolos padrão, como o HTTP, para oferecer acesso a recursos, que são representados por URLs únicas. Os recursos podem ser manipulados por meio de métodos HTTP como GET (para obter dados), POST (para criar novos dados), PUT (para atualizar dados existentes) e DELETE (para remover dados). Além disso, uma REST API geralmente retorna respostas no formato JSON ou XML, facilitando a integração com aplicações diversas, incluindo sistemas web, mobile ou IoT. Por sua simplicidade e flexibilidade, REST APIs são amplamente utilizadas para construir sistemas escaláveis e interoperáveis.

1.3. Git

Git é um sistema de controle de versão distribuído amplamente utilizado para gerenciar projetos de desenvolvimento de software. Ele permite que os desenvolvedores acompanhem mudanças no código ao longo do tempo, colaborem de forma eficiente e revertam alterações, se necessário. Com Git, o código-fonte é armazenado em repositórios, podendo ser acessado e modificado por vários contribuidores simultaneamente. Suas principais características incluem ramificação e mesclagem, que facilitam o trabalho em diferentes funcionalidades ou correções sem interferir na base principal do projeto. Além disso, o Git é rápido, seguro e funciona tanto localmente quanto em servidores remotos, como GitHub e GitLab, tornando-o uma ferramenta essencial para equipes de desenvolvimento modernas.

1.4. GitHub

GitHub é uma plataforma baseada na web que utiliza o Git como sistema de controle de versão para gerenciar e hospedar repositórios de código-fonte. Além de ser um repositório remoto, o GitHub oferece diversas ferramentas colaborativas, como **pull requests** (para sugerir e revisar alterações no código), **issues** (para rastrear problemas ou tarefas), e wikis (para documentar projetos). Ele também integra funcionalidades de CI/CD (Integração e Entrega Contínuas), que automatizam testes e implantações. Amplamente usado por desenvolvedores e organizações, o GitHub suporta tanto projetos de código aberto quanto privados, permitindo colaboração global. Sua interface intuitiva, aliada à ampla comunidade de usuários, faz dele um dos principais ambientes para compartilhar, revisar e contribuir para projetos de software.

1.4.1. Actions

GitHub Actions é uma ferramenta de automação integrada ao GitHub que permite criar e executar fluxos de trabalho personalizados diretamente nos repositórios. Com ela, os desenvolvedores podem automatizar tarefas como testes, compilações, implantações e muito mais, usando um arquivo de configuração YAML. Esses fluxos de trabalho são acionados por eventos no repositório, como pushes, pull requests, criação de tags ou agendamentos.

As ações são compostas por etapas (steps), que podem executar comandos ou utilizar ações pré-definidas disponíveis no marketplace do GitHub. Além disso, o GitHub Actions oferece suporte a múltiplos sistemas operacionais, linguagens de programação e ambientes, permitindo testes em matrizes configuráveis. Por ser altamente flexível, ele é amplamente usado para implementar pipelines de CI/CD (Integração e Entrega Contínuas), facilitando o desenvolvimento, a colaboração e o lançamento de software.

1.4.2. Runners

Ele é responsável por processar as tarefas descritas nos arquivos de configuração YAML, como compilar código, rodar testes e realizar implantações. Existem dois tipos principais de runners no GitHub:

1. **Runners hospedados pelo GitHub:** São gerenciados diretamente pela plataforma e fornecem ambientes pré-configurados para sistemas operacionais como Linux, Windows e macOS. Esses runners são ideais para a maioria dos projetos, pois eliminam a necessidade de configuração manual e oferecem escalabilidade automática.
2. **Self-hosted runners:** São runners configurados pelos próprios usuários em suas máquinas ou servidores. Eles oferecem maior controle, permitindo personalizar o ambiente de execução com softwares específicos ou hardware dedicado, sendo úteis para cenários que exigem requisitos especiais ou sensíveis.

Os runners recebem as tarefas em fila do GitHub Actions, baixam o código do repositório, executam os comandos especificados no fluxo de trabalho e retornam os resultados para o

GitHub. Com essa abordagem, o GitHub combina flexibilidade e eficiência na execução de automações.

1.5. Docker

Docker é uma plataforma que permite criar, distribuir e executar aplicativos em contêineres. Um contêiner é uma unidade leve e portátil que encapsula um aplicativo e todas as suas dependências (bibliotecas, configurações, etc.), garantindo que ele funcione de forma consistente em diferentes ambientes, seja na máquina do desenvolvedor, em servidores de produção ou na nuvem.

Os contêineres são gerenciados pelo Docker Engine, que utiliza tecnologias de virtualização no nível do sistema operacional para garantir desempenho eficiente. Com o Docker, é possível criar imagens, que são "modelos" imutáveis usados para instanciar contêineres. Essas imagens são definidas por meio de um **Dockerfile**, que especifica as instruções para construir o ambiente necessário.

Docker é amplamente utilizado para melhorar o fluxo de trabalho de desenvolvimento, simplificar a implantação de aplicativos e facilitar a escalabilidade. Ele também integra ferramentas como o Docker Compose (para gerenciar múltiplos contêineres) e o Docker Hub (um repositório para compartilhar e baixar imagens). Ao padronizar ambientes, o Docker reduz problemas de compatibilidade e acelera a entrega de software.

1.5.1 DockerHub

Docker Hub é um repositório online para armazenar, compartilhar e gerenciar imagens de contêineres Docker. Ele permite que desenvolvedores publiquem imagens personalizadas, acessem uma vasta biblioteca de imagens públicas, incluindo versões oficiais de softwares populares, e armazenem imagens privadas para projetos específicos. Com integração a ferramentas como GitHub, o Docker Hub também suporta pipelines automáticos para criar e atualizar imagens. Além disso, facilita a colaboração em equipe e a distribuição global de contêineres, tornando o desenvolvimento e a implantação de aplicações mais simples e eficientes.

1.6. AWS

AWS (Amazon Web Services) é uma plataforma de computação em nuvem fornecida pela Amazon que oferece uma ampla gama de serviços para empresas e desenvolvedores. Esses serviços incluem computação, armazenamento, bancos de dados, redes, inteligência artificial, análise de dados, segurança, e muito mais, todos acessíveis pela internet. A AWS permite criar, escalar e gerenciar aplicações de forma flexível, com pagamento baseado no uso, sem a necessidade de gerenciar infraestrutura física. Amplamente utilizada em setores diversos, a AWS é reconhecida por sua confiabilidade, escalabilidade e vasto ecossistema, sendo uma das principais plataformas de computação em nuvem do mundo.

1.6.1. EC2

O Amazon EC2 (Elastic Compute Cloud) é um serviço da AWS que fornece capacidade de computação escalável na nuvem. Ele permite que os usuários criem e gerenciem instâncias de servidores virtuais, chamadas de **instâncias EC2**, que podem ser configuradas com diferentes sistemas operacionais, tamanhos de recursos (CPU, memória e armazenamento) e redes.

O EC2 é projetado para ser altamente flexível, permitindo aumentar ou reduzir a capacidade conforme a demanda, o que é ideal para aplicações de diferentes tamanhos, de pequenas startups a grandes corporações. Além disso, ele suporta diversos modelos de pagamento, como **On-Demand** (pagamento pelo uso), **Reserved Instances** (custo reduzido para uso contínuo) e **Spot Instances** (custo mais baixo para cargas de trabalho flexíveis).

Com o EC2, os desenvolvedores podem implantar, escalar e gerenciar aplicativos rapidamente, sem precisar investir em hardware físico, garantindo eficiência e custo-benefício.

1.7. CI/CD

CI/CD (Integração Contínua e Entrega/Implantação Contínua) é um conjunto de práticas que visa melhorar a eficiência e a qualidade no desenvolvimento de software, automatizando os processos de integração e entrega de código. A Integração Contínua (CI) envolve a prática de integrar o código de todos os desenvolvedores em um repositório central várias vezes ao dia. Cada integração é verificada por meio de testes automatizados, o que ajuda a identificar e corrigir erros rapidamente. Isso permite que as equipes detectem problemas de integração de forma precoce, melhorando a colaboração entre os desenvolvedores e evitando que erros se acumulem.

A Entrega Contínua (CD) e Implantação Contínua (CD) são práticas que seguem a CI e se concentram na automação do processo de implantação do código em diferentes ambientes. A Entrega Contínua garante que o código esteja sempre pronto para ser implantado em produção, após passar pelos testes, mas a implantação em si pode ser feita manualmente. Já a Implantação Contínua leva a automação um passo adiante, realizando a implantação automaticamente em produção assim que o código passa nos testes. Essas práticas ajudam as equipes a entregar software de forma mais rápida, com maior qualidade e menos riscos, tornando o processo de desenvolvimento mais ágil e previsível.

2. O Projeto

O objetivo deste projeto é criar uma pipeline de Integração Contínua e Deploy Contínuo (CI/CD) para uma aplicação Go, utilizando Docker e GitHub Actions para automatizar o processo de build e deploy. Através da criação de um arquivo Dockerfile, a aplicação Go será empacotada em um container, garantindo que o ambiente de execução seja consistente em diferentes máquinas. Com o uso do GitHub Actions, serão definidos

workflows para monitorar mudanças no repositório, realizando automaticamente o build da aplicação, a criação da imagem Docker e o deploy para o ambiente de produção ou staging. Esse processo automatizado visa aumentar a eficiência e reduzir erros manuais, permitindo que novas versões da aplicação sejam entregues de forma rápida e segura.

3. Passo a passo

1. Pré-requisitos
2. Aplicação go
3. Dockerização
4. Criação do Repositório GitHub
5. Segredo no GitHub
6. Criar Repositório Dockerhub
7. Preparar VM
8. Self-hosted Runner
9. Action CI
10. Action CD

3.1. Pré-requisitos

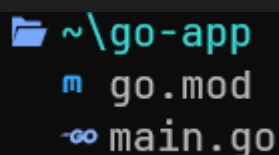
- Golang instalado na maquina local
- Um editor de sua preferência
- Docker instalado na máquina local
- Conta no Docker Hub
- Conta na AWS
- Conta no GitHub

3.2. Aplicação Go

- Iniciar uma aplicação Go;

```
mkdir go-app
cd go-app
go mod init go-app
```

- Sua pasta deve estar assim:



```
~\go-app
├── go.mod
└── main.go
```

- Criar um arquivo *main.go* na raiz do projeto;


```
package main

import (
    "fmt"
    "net/http"
)

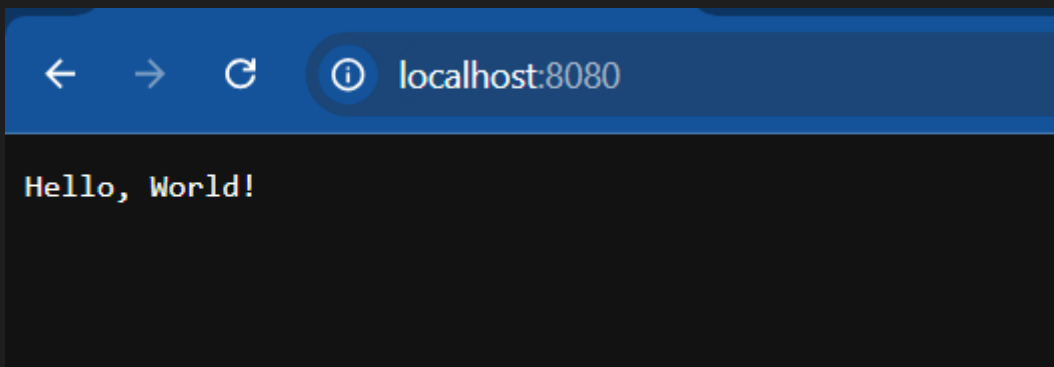
func handler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, World!")
}

func main() {
    http.HandleFunc("/", handler)
    fmt.Println("Starting server on :8080")
    if err := http.ListenAndServe(":8080", nil); err != nil {
        fmt.Println("Error starting server:", err)
    }
}
```

- Rodar o programa

```
go run .
```

- Testar no navegador ou com curl



```
curl "http://localhost:8080"
```

```
PS C:\Users\joaov> curl "http://localhost:8080"
Hello, World!
```

3.3. Dockerização

- Criar um Dockerfile na raiz do projeto

```
FROM golang:1.23-alpine AS builder

WORKDIR /app

COPY go.* ./

RUN go mod tidy

COPY . .

RUN go build -o main .

FROM alpine:latest

RUN apk --no-cache add ca-certificates

WORKDIR /root/

COPY --from=builder /app/main .

EXPOSE 8080

CMD ["/main"]
```

Para outras linguagens

Este passo é o único que depende da linguagem, caso esteja utilizando outra linguagem neste tutorial, deve-se utiliza um Dockerfile diferente.

- Criar um .dockerignore na raiz do projeto

```
.gitignore
.git
.github
```

.dockerignore

O .dockerignore funciona da mesma forma que o .gitignore, os arquivos listados nele não serão copiados para o container, o uso básico é colocar tudo que está presente no .gitignore e os arquivos relacionados a versionamento e desenvolvimento e deixar de fora tudo que seja relacionado a produção

- Testar dockerfile localmente

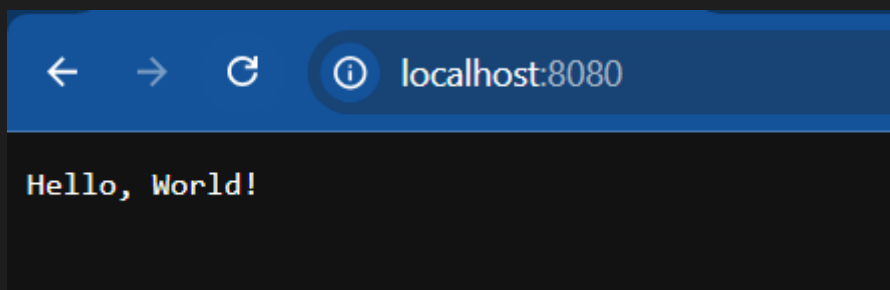
```
docker build -t go-app .
```

```
docker run -p 8080:8080 --name go-app-container go-app
```

```
PS C:\Users\joaov\go-app> docker build -t go-app .
[+] Building 43.2s (18/18) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 314B
=> [internal] load metadata for docker.io/library/alpine:latest
=> [internal] load metadata for docker.io/library/golang:1.23-alpine
=> [auth] library/alpine:pull token for registry-1.docker.io
=> [auth] library/golang:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 67B
=> [builder 1/6] FROM docker.io/library/golang:1.23-alpine@sha256:9a31ef0803e6afdf564edc8ba4b4e17caed22a0b1ecd2c55e3c8fdd8d8f68f98
=> => resolve docker.io/library/golang:1.23-alpine@sha256:9a31ef0803e6afdf564edc8ba4b4e17caed22a0b1ecd2c55e3c8fdd8d8f68f98
=> => sha256:5fe327b54edc2534d4a94dc17716d2b1655e6cf233f86390c8a9b854885ae545 125B / 125B
=> => sha256:06f05ace1117d62b655e04f6f73c83617e3e0feb38681dbedf58f477dd0658c 74.05MB / 74.05MB
=> => sha256:6ef4a4ee8ca5db55a7139577107921417e5e27350f34f39667f17d271f9dbd1b 290.89kB / 290.89kB
=> => extracting sha256:6ef4a4ee8ca5db55a7139577107921417e5e27350f34f39667f17d271f9dbd1b
=> => extracting sha256:06f05ace1117d62b655e04f6f73c83617e3e0feb38681dbedf58f477dd0658c
=> => extracting sha256:5fe327b54edc2534d4a94dc17716d2b1655e6cf233f86390c8a9b854885ae545
=> => extracting sha256:4f4fb708ef54461cfa02571ae0db9a0dc1e0cdb55774846d75e68dc38e8acc1
=> [internal] load build context
=> => transferring context: 830B
=> [stage-1 1/4] FROM docker.io/library/alpine:latest@sha256:21dc6063fd678b478f57c0e13f47560d0ea4eeba26dfc947b2a4f81f686b9f45
=> => resolve docker.io/library/alpine:latest@sha256:21dc6063fd678b478f57c0e13f47560d0ea4eeba26dfc947b2a4f81f686b9f45
=> => sha256:38a8318d387e375e0ec6fabe047a9149e8eb214073db9f461fee6251fd936a75 3.64MB / 3.64MB
=> => extracting sha256:38a8318d387e375e0ec6fabe047a9149e8eb214073db9f461fee6251fd936a75
=> [stage-1 2/4] RUN apk --no-cache add ca-certificates
=> [builder 2/6] WORKDIR /app
=> [stage-1 3/4] WORKDIR /root/
=> [builder 3/6] COPY go.* ./
=> [builder 4/6] RUN go mod tidy
=> [builder 5/6] COPY . .
=> [builder 6/6] RUN go build -o main .
=> [stage-1 4/4] COPY --from=builder /app/main .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:f5fe60d2ca257c394324553cccfdf3f996b875684c181ff15610c556b6c8cd89
=> => exporting config sha256:37b8d9b6f4f59cad67002b3a7d80b0367c2be1ebc016546455e2dd6556fc5e53
=> => exporting attestation manifest sha256:fa234fa1526cd1d879fba7f5e6292036260b079373f78278742ffaace26519d
=> => exporting manifest list sha256:d16d79d3b47f71bf5e02230597ec9e85937980f6d16221f80785bb44c388141
=> => naming to docker.io/library/go-app:latest
=> => unpacking to docker.io/library/go-app:latest
```

```
PS C:\Users\joaov\go-app> docker run -p 8080:8080 --name go-app-container go-app
Starting server on :8080
```

- Testar no navegador ou com curl!



```
curl "http://localhost:8080"
```

```
PS C:\Users\joaov> curl "http://localhost:8080"
Hello, World!
```

3.4. Criação do Repositório no GitHub

- Na página inicial do GitHub, clique em "New"

Top repositories

[New](#)

- Escolher um nome para o repositório

Owner *

 joaobgoode ▾

Repository name *

/ ckd-pipeline

✓ ckd-pipeline is available.

Great repository names are short and memorable. Need inspiration? How about **fluffy-lamp** ?

Description (optional)



Public

Anyone on the internet can see this repository. You choose who can commit.



Private

You choose who can see and commit to this repository.

Boas práticas!

Aqui, por simplicidade, foi utilizado o mesmo nome para repositório do GitHub, DockerHub e container, é importante manter um padrão, não precisa ser tudo igual, mas é importante ter consistência para evitar confusão.

- Para este tutorial, manter o repositório publico

Por que público?

O GitHub impõe limitações em Actions para repositórios privados!

- Na raiz do projeto, abrir um terminal e rodar os comandos:

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin LINK_PARA_O_REPOSITÓRIO
git push -u origin main
```

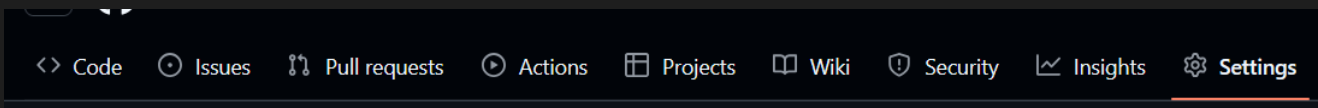
```

PS C:\Users\joaov\go-app> git init
Initialized empty Git repository in C:/Users/joaov/go-app/.git/
PS C:\Users\joaov\go-app> git add .
warning: in the working copy of 'go.mod', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\joaov\go-app> git commit -m "first commit"
[master (root-commit) 395c114] first commit
4 files changed, 47 insertions(+)
create mode 100644 .dockerignore
create mode 100644 Dockerfile
create mode 100644 go.mod
create mode 100644 main.go
PS C:\Users\joaov\go-app> git branch -M main
PS C:\Users\joaov\go-app> git remote add origin https://github.com/joaobgoode/cicd-pipeline.git
PS C:\Users\joaov\go-app> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 792 bytes | 792.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/joaobgoode/cicd-pipeline.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

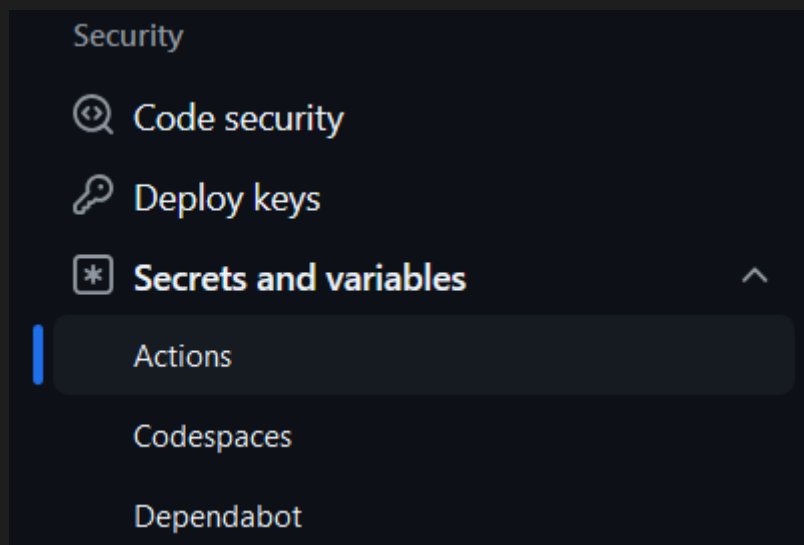
```

3.5. Segredo no GitHub

- Na página do repositório no GitHub, clicar em "Settings"

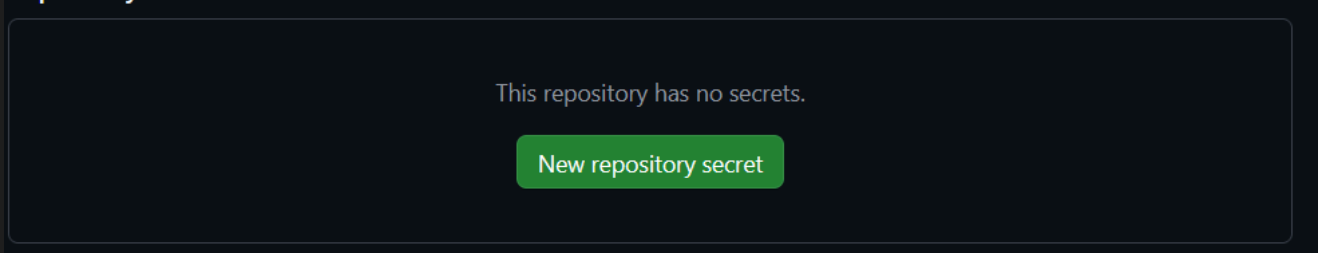


- No menu lateral, procurar por "Secrets and variables"
- Clicar em "Secrets and variables" e depois em "Actions"
-



- Clicar em "New repository secret"

Repository secrets



- Adicionar os seguintes segredos:

DOCKER_USERNAME
DOCKER_PASSWORD

Repository secrets			New repository secret
Name 	Last updated		
 DOCKER_PASSWORD	now		
 DOCKER_USERNAME	now		

⚠ Cuidado!

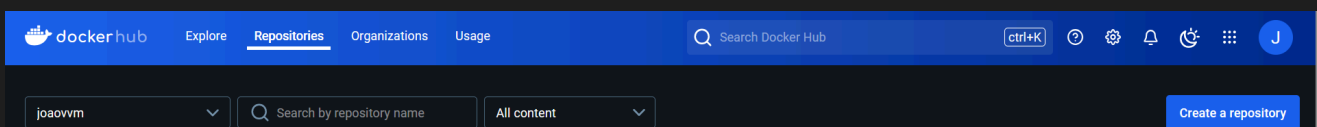
Colocar seu usuário com letras minúsculas apenas

3.6. Criar Repositório no Docker Hub

🔗 Passo Opcional!

O script da action irá criar o repositório no DockerHub caso não exista!

- Na página inicial do DockerHub clique em "Create a repository"



- Adicionar nome do repositório
- Adicionar descrição

Create repository

Namespace
joavvm

Repository Name *
cicd-pipeline


Short description
tutorial cicd-pipeline

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Get more](#)

☒ Public 
Appears in Docker Hub search results

☐ Private 
Only visible to you

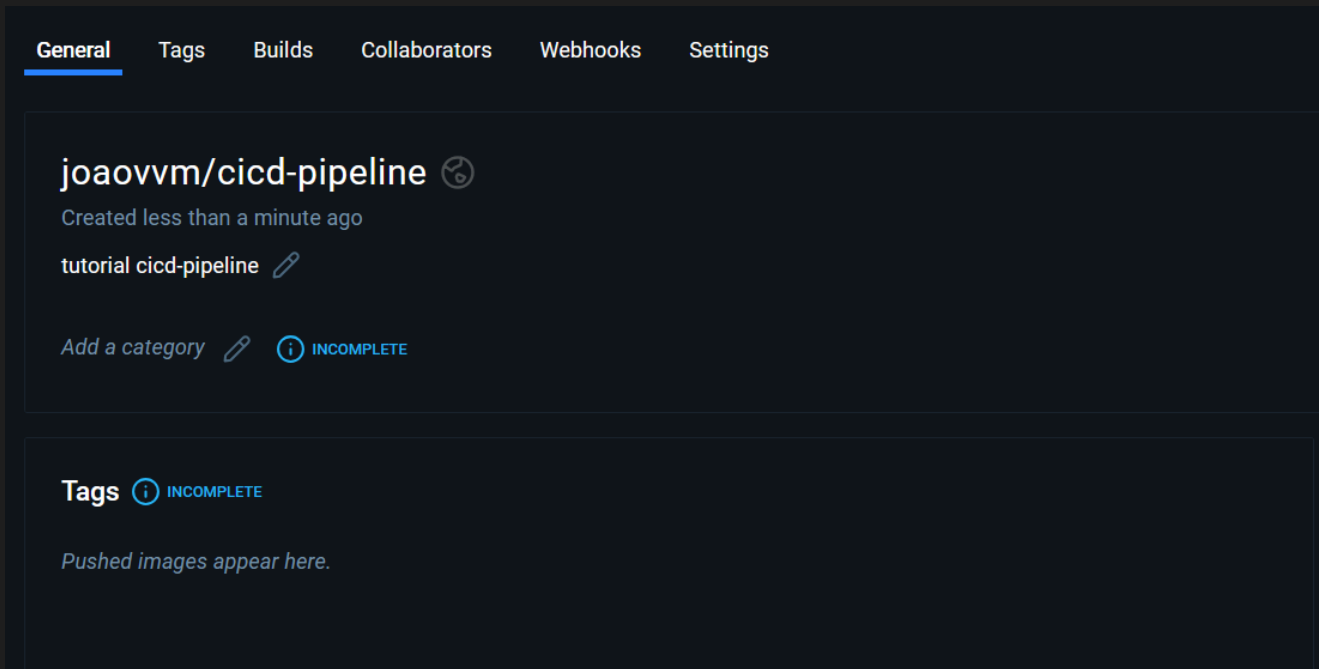
Cancel
Create

- Deixar publico

❓ Público denovo?

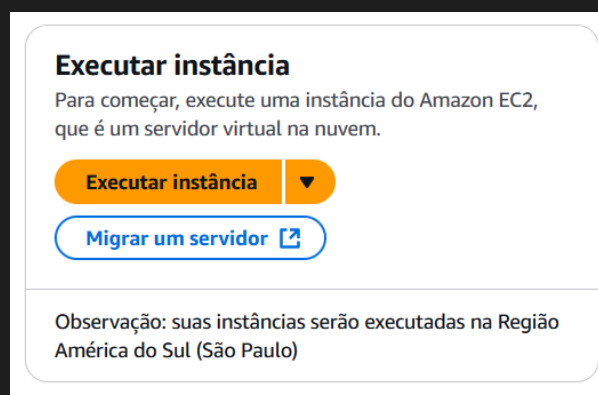
Sim, o DockerHub limita ainda mais repositórios privados, uma conta grátis só pode ter um repositório privado, além disso um usuário grátis está limitado a 200 pulls / 6 horas, o que é mais do que suficiente para projetos pequenos.

- Clicar em create



3.7. Preparar VM

- Criar instancia



⚠️ Outros provedores de nuvem

Este tutorial funciona independente do provedor, caso não esteja usando Oracle, faça o equivalente deste passo para o provedor escolhido

- Escolher imagem

▼ Imagens de aplicação e de sistema operacional (imagem de máquina da Amazon) [Informações](#)

Uma AMI é um modelo que contém a configuração do software (sistema operacional, servidor de aplicações e aplicações) necessária para executar a instância. Pesquise ou navegue pelas AMIs se você não estiver vendo o que está buscando abaixo

Pesquise nosso catálogo completo, incluindo milhares de imagens de aplicações e sistemas operacionais

Início rápido

Amazon Linux


Ubuntu


Windows


Red Hat


SUSE Linux


Debian



[Procurar mais AMIs](#)
Incluindo AMIs da AWS, do Marketplace e da comunidade

Imagem de máquina da Amazon (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-015f3596bb2ef1aaa (64 bits (x86)) / ami-044ed978932f3d06a (64 bits (Arm))
Virtualização: hvm ENA habilitado: true Tipo de dispositivo raiz: ebs

Qualificado para o nível gratuito ▼

⚠ Outras distribuições linux

Neste tutorial foi utilizado Ubuntu, verificar os passos caso estiver utilizando outra distribuição na [documentação oficial](#)

- Liberar tráfego

Firewall (grupos de segurança) [Informações](#)

Um grupo de segurança é um conjunto de regras de firewall que controlam o tráfego para sua instância. Adicione regras para permitir que o tráfego específico alcance sua instância.

☒ Criar grupo de segurança

☐ Selecionar grupo de segurança existente

Criaremos um novo grupo de segurança chamado **"launch-wizard-1"** com as seguintes regras:

- ☒ Permitir tráfego SSH de
Ajuda você a se conectar à sua instância
- ☒ Permitir tráfego HTTPS da Internet
Para configurar um endpoint, por exemplo, ao criar um servidor Web
- ☒ Permitir tráfego HTTP da Internet
Para configurar um endpoint, por exemplo, ao criar um servidor Web

Qualquer lugar
0.0.0.0/0 ▼

⚠ Regras com origem 0.0.0.0/0 permitem que todos os endereços IP acessem sua instância. Recomendamos configurar regras de grupo de segurança para permitir o acesso apenas de endereços IP conhecidos.

- Executar instância



Êxito

Execução da instância iniciada com êxito (i-0695df172b3076134)

► Log de execução

Próximas etapas

- Clicar no nome da instância
- Clicar em "Segurança"

i-0695df172b3076134

Detalhes | Status e alarmes | Monitoramento | **Segurança** | Redes | Armazenamento | Tags

- Clicar no grupo de segurança

Grupos de segurança

 [sg-02836f9cb33784bf4 \(launch-wizard-1\)](#)

- Clicar em "Editar regras de entrada"

Regras de entrada (3)



Gerenciar tags

Editar regras de entrada

Q Pesquisar

< 1 > ⚙

☐	Name	ID da regra do grup...	Versão do IP	Tipo	Protocolo	Intervalo de portas
☐	-	sgr-0c410597aea1de46f	IPv4	HTTP	TCP	80
☐	-	sgr-09355bc2e34c416ee	IPv4	HTTPS	TCP	443
☐	-	sgr-0589ba5a9ea8d1990	IPv4	SSH	TCP	22

- Clicar em "Adicionar regra"

sgr-0c410597aea1de46f	HTTP	TCP	80	Per...	Q		Excluir
					0.0.0.0/0 X		
sgr-09355bc2e34c416ee	HTTPS	TCP	443	Per...	Q		Excluir
					0.0.0.0/0 X		
sgr-0589ba5a9ea8d1990	SSH	TCP	22	Per...	Q		Excluir
					0.0.0.0/0 X		
Adicionar regra							

- Liberar a porta utilizada (no caso a 8080) com origem IPv4

TCP personalizado ▼ TCP 8080 Qu... 0.0.0.0/0 X Excluir

- Salvar alterações

Recomendamos configurar as regras de grupo de segurança para permitir o acesso X

Cancelar Visualizar alterações Salvar regras

- Clicar em instâncias

▼ **Instâncias**

Instâncias

Tipos de instância

Modelos de execução

Solicitações spot

- Clicar no ID da instância

☒	Name ✎ ▼	ID da instância	Estado da inst... ▼
☒		i-0695df172b3076134	✔ Executando 🔍 🔍

- Conectar à VM

🔄 Conectar Estado da instância ▼ Ações ▼

ciar a instância. Se você não definiu um nome de usuário personalizado, use o nome de usuário padrão, ubuntu.

e de usuário padrão, ubuntu, está correto. No entanto, leia as instruções de uso da AMI para verificar se o proprietário da AMI alterou o nome de

Cancelar

Conectar

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

```
ubuntu@ip-172-31-46-184:~$ █
```

- Instalar Docker

[Seguir os passos nos docs oficiais](#)

1. Adiciona a GPG key do Docker

```
sudo apt-get update && \
sudo apt-get install ca-certificates curl && \
sudo install -m 0755 -d /etc/apt/keyrings && \
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc && \
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```

Get:36 http://sa-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 c-n-f Metadata [116 B]
Get:37 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [498 kB]
Get:38 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [102 kB]
Get:39 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [7188 B]
Get:40 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [563 kB]
Get:41 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [150 kB]
Get:42 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [51.9 kB]
Get:43 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [13.5 kB]
Get:44 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [480 kB]
Get:45 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [92.5 kB]
Get:46 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:47 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [12.2 kB]
Get:48 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [2940 B]
Get:49 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:50 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [356 B]
Fetched 30.9 MB in 13s (2468 kB/s)
sudo apt-get install ca-certificates curl
Reading package lists... Done
ubuntu@ip-172-31-46-184:~$ ^[[200~sudo install -m 0755 -d /etc/apt/keyrings~^C
ubuntu@ip-172-31-46-184:~$ sudo install -m 0755 -d /etc/apt/keyrings
ubuntu@ip-172-31-46-184:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
ubuntu@ip-172-31-46-184:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc

```

2. Adiciona o Docker à lista do APT

```

echo \ "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \ sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null && \
sudo apt-get update

```

```

ubuntu@ip-172-31-46-184:~$ echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
ubuntu@ip-172-31-46-184:~$ sudo apt-get update
Hit:1 http://sa-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://sa-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://sa-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:4 https://download.docker.com/linux/ubuntu noble InRelease [48.8 kB]
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [15.7 kB]
Fetched 64.5 kB in 1s (96.0 kB/s)
Reading package lists... Done
ubuntu@ip-172-31-46-184:~$ █

```

3. Instale o Docker

```

sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

```

```

ubuntu@ip-172-31-46-184:~$ sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras libltdl7 libsllp0 pigz sllrp4netns
Suggested packages:
  aufs-tools cgroupfs-mount | cgroup-lite
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin libltdl7 libsllp0 pigz sllrp4netns
0 upgraded, 10 newly installed, 0 to remove and 40 not upgraded.
Need to get 123 MB of archives.
After this operation, 442 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://sa-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 http://sa-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libltdl7 amd64 2.4.7-7build1 [40.3 kB]
Get:3 http://sa-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libsllp0 amd64 4.7.0-1ubuntu3 [63.8 kB]
Get:4 http://sa-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 sllrp4netns amd64 1.2.1-1build2 [34.9 kB]
Get:5 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 1.7.24-1 [29.5 MB]
Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-buildx-plugin amd64 0.17.1-1ubuntu.24.04~noble [30.3 MB]
Get:7 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-cli amd64 5:27.3.1-1ubuntu.24.04~noble [15.0 MB]
Get:8 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:27.3.1-1ubuntu.24.04~noble [136.6 MB]

```



```
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-46-184:~$
```

4. Verifique se a instalação foi concluída com sucesso:

```
sudo docker run hello-world
```

```
ubuntu@ip-172-31-46-184:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest: sha256:305243c734571da2d100c8c8b3c3167a098cab6049c9a5b066b6021a60fcb966
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/
```

- Faça log in no Docker

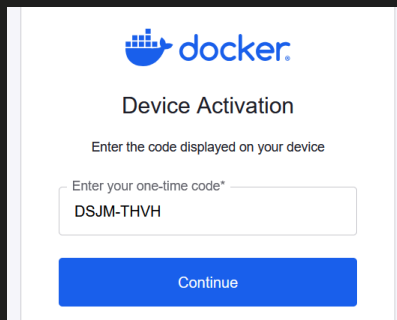
```
docker login
```

- Seguir os passos indicados no console

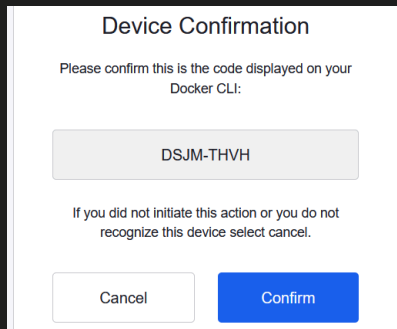
```
ubuntu@ip-172-31-46-184:~$ docker login

USING WEB-BASED LOGIN
To sign in with credentials on the command line, use 'docker login -u <username>'

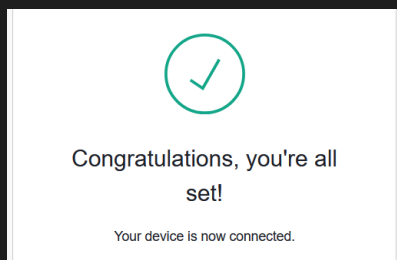
Your one-time device confirmation code is: DSJM-THVH
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate
```



The Docker Device Activation screen features the Docker logo at the top. Below it, the title "Device Activation" is centered. A subtitle reads "Enter the code displayed on your device". A text input field is labeled "Enter your one-time code*" and contains the code "DSJM-THVH". A blue "Continue" button is positioned at the bottom.



The Docker Device Confirmation screen has the title "Device Confirmation". The subtitle says "Please confirm this is the code displayed on your Docker CLI:". A grey button displays the code "DSJM-THVH". Below this, a warning message states: "If you did not initiate this action or you do not recognize this device select cancel." At the bottom are two buttons: a grey "Cancel" button and a blue "Confirm" button.



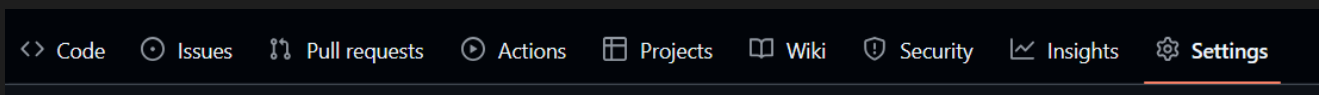
The Docker Success screen features a large green checkmark icon inside a circle. Below the icon, the text reads "Congratulations, you're all set!". At the bottom, a smaller line of text says "Your device is now connected."

```
Waiting for authentication in the browser...
WARNING! Your password will be stored unencrypted in /home/ubuntu/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

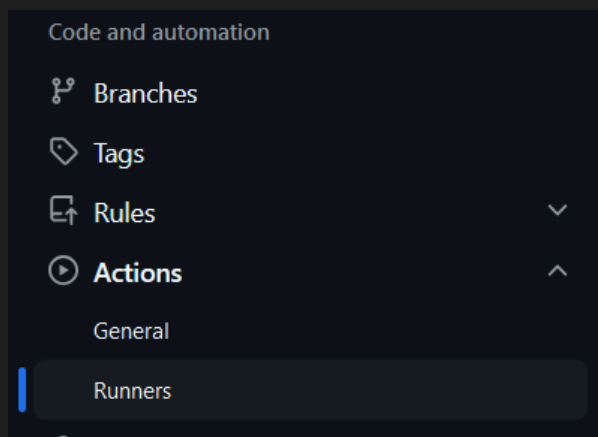
Login Succeeded
ubuntu@ip-172-31-46-184:~$
```

3.8. Self Hosted Runner

- Na página do repositório, clicar em Settings



- No menu lateral clicar em "Actions" na parte de "Code and automation" e depois em "Runners"



- Clicar em "New self-hosted runner"

Runners

Host your own runners and customize the environment used to run jobs in your GitHub Actions workflows. [Learn more about self-hosted runners.](#)

New self-hosted runner

- Selecionar linux e a arquitetura da VM

Runner image

☐ macOS
 ☒ Linux
 ☐ Windows

Architecture

x64

- Seguir os passos indicados pelo próprio GitHub, usando os comandos na parte de "Download" e "Configure" na VM um de cada vez

```
ubuntu@ip-172-31-46-184:~$ mkdir actions-runner && cd actions-runner
ubuntu@ip-172-31-46-184:~/actions-runner$ curl -o actions-runner-linux-x64-2.321.0.tar.gz -L https://github.com/actions/runner/releases/download/v2.321.0/actions-runner-linux-x64-2.321.0.tar.gz
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0:00:02 0:00:02 --:--:-- 105M
ubuntu@ip-172-31-46-184:~/actions-runner$ echo "ba46ba7ce3a4d7236b16f6e4419fb453bc08f866b24f04d549ec89f1722a29e actions-runner-linux-x64-2.321.0.tar.gz" | shasum -a 256 -c
actions-runner-linux-x64-2.321.0.tar.gz: OK
ubuntu@ip-172-31-46-184:~/actions-runner$ tar xzf ./actions-runner-linux-x64-2.321.0.tar.gz

ubuntu@ip-172-31-46-184:~/actions-runner$ ./config.sh --url https://github.com/joaobgoode/cicd-pipeline --token BHY74ZWZM3BH4QXMC7UNEJTHKML3C

-----
GitHub Actions
Self-hosted runner registration
-----

# Authentication

✓ Connected to GitHub

# Runner Registration

Enter the name of the runner group to add this runner to: [press Enter for Default]
Enter the name of runner: [press Enter for ip-172-31-46-184]
```

- Verifique que funcionou

```
./run.sh
```

```
ubuntu@ip-172-31-46-184:~/actions-runner$ ./run.sh
✓ Connected to GitHub

Current runner version: '2.321.0'
2024-12-06 14:30:30Z: Listening for Jobs
□
```

- A página dos Runners no repositório deve constar o Runner que você acabou de configurar

Runners

Host your own runners and customize the environment used to run jobs in your GitHub Actions workflows. [Learn more about self-hosted runners.](#)

Runners	Status
 ip-172-31-46-184 self-hosted Linux X64	● Idle ...

- Pare a execução com ctrl+c

```
2024-12-06 14:30:30Z: Listening for Jobs
^CExiting...
Runner listener exit with 0 return code, stop the service, no retry needed.
Exiting runner...
ubuntu@ip-172-31-46-184:~/actions-runner$ □
```

- Inicie o Runner de forma persistente

```
sudo ./svc.sh install
sudo ./svc.sh start
```

```
/etc/systemd/system/actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service
● actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service - GitHub Actions Runner (joaobgoode-cicd-pipeline.ip-172-31-46-184)
   Loaded: loaded (/etc/systemd/system/actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service; enabled; preset: enabled)
   Active: active (running) since Fri 2024-12-06 14:32:39 UTC; 15ms ago
     Main PID: 3075 (runsv.sh)
       Tasks: 2 (limit: 1130)
      Memory: 372.0K (peak: 832.0K)
         CPU: 3ms
    CGroup: /system.slice/actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service
            └─3075 /bin/bash /home/ubuntu/actions-runner/runsv.sh
              └─3078 ./externals/node20/bin/node ./bin/RunnerService.js

Dec 06 14:32:39 ip-172-31-46-184 systemd[1]: Started actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service - GitHub Actions Runner (joaobgoode-ci...-31-46-184).
Dec 06 14:32:39 ip-172-31-46-184 runsv.sh[3075]: .path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@ip-172-31-46-184:~/actions-runner$ □
```

- Checar o status do Runner

```
sudo ./svc.sh status
```

```
ubuntu@ip-172-31-46-184:~/actions-runner$ sudo ./svc.sh status
/etc/systemd/system/actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service
● actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service - GitHub Actions Runner (joaobgoode-cicd-pipeline.ip-172-31-46-184)
   Loaded: loaded (/etc/systemd/system/actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service; enabled; preset: enabled)
   Active: active (running) since Fri 2024-12-06 14:32:39 UTC; 38s ago
     Main PID: 3075 (runsv.sh)
       Tasks: 20 (limit: 1130)
    Memory: 75.8M (peak: 76.1M)
       CPU: 1.062s
    CGroup: /system.slice/actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service
            └─3075 /bin/bash /home/ubuntu/actions-runner/runsv.sh
            └─3078 ./externals/node20/bin/node ./bin/RunnerService.js
            └─3085 /home/ubuntu/actions-runner/bin/Runner.Listener run --startuptype service

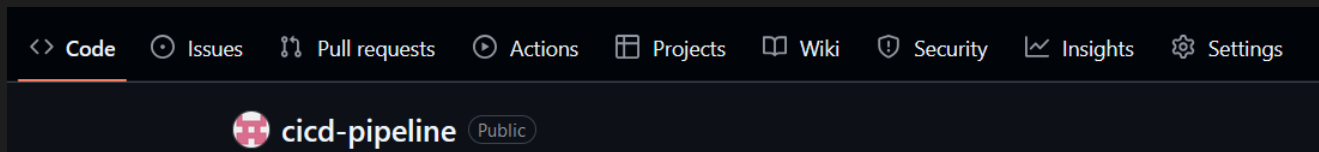
Dec 06 14:32:39 ip-172-31-46-184 systemd[1]: Started actions.runner.joaobgoode-cicd-pipeline.ip-172-31-46-184.service - GitHub Actions Runner (joaobgoode-ci...-31-46-184).
Dec 06 14:32:39 ip-172-31-46-184 runsv.sh[3075]: .path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
Dec 06 14:32:39 ip-172-31-46-184 runsv.sh[3078]: Starting Runner listener with startup type: service
Dec 06 14:32:39 ip-172-31-46-184 runsv.sh[3078]: Started listener process, pid: 3085
Dec 06 14:32:39 ip-172-31-46-184 runsv.sh[3078]: Started running service
Dec 06 14:32:42 ip-172-31-46-184 runsv.sh[3078]: √ Connected to GitHub
Dec 06 14:32:43 ip-172-31-46-184 runsv.sh[3078]: Current runner version: '2.321.0'
Dec 06 14:32:43 ip-172-31-46-184 runsv.sh[3078]: 2024-12-06 14:32:43Z: Listening for Jobs
Hint: Some lines were ellipsized, use -l to show in full.
```

🔥 **Dica: Para parar o Runner basta parar o serviço**

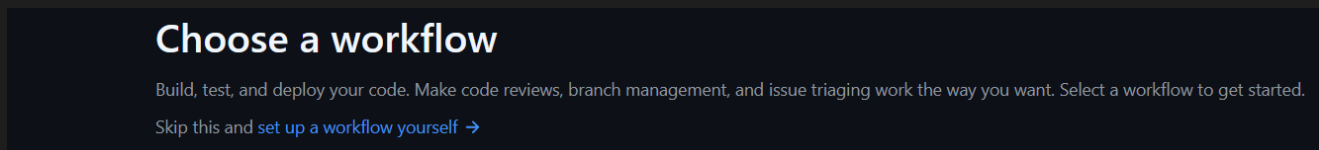
```
sudo ./svc.sh stop
```

3.9. Action CI

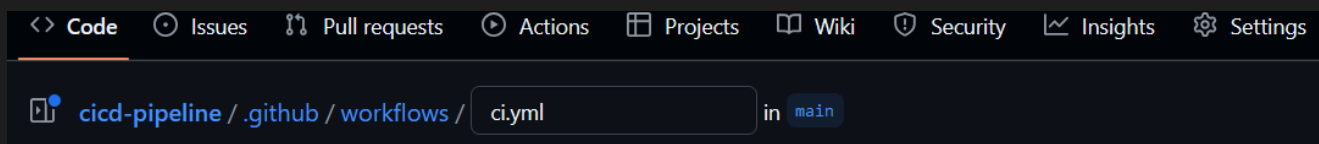
- Na página do repositório clicar em "Actions"



- Clicar em "Set up a workflow yourself"



- Trocar o nome para "*ci.yml*"



- Colar o código abaixo:

```
name: Docker Image CI

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

```

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
      - name: Log in to DockerHub
        run: echo ${ secrets.DOCKER_PASSWORD }} | docker login -u ${ secrets.DOCKER_USERNAME }} --password-stdin
      - name: Build and Push Docker image
        run: |
          IMAGE_TAG=${ github.sha }}
          docker build -t ${ secrets.DOCKER_USERNAME }}/cicd-pipeline:latest -t ${ secrets.DOCKER_USERNAME }}/cicd-pipeline:$IMAGE_TAG .
          docker push ${ secrets.DOCKER_USERNAME }}/cicd-pipeline:latest
          docker push ${ secrets.DOCKER_USERNAME }}/cicd-pipeline:$IMAGE_TAG

```


cicd-pipeline / .github / workflows / ci.yml in main

EditPreview

Spaces2No wrap

```

1  name: Docker Image CI
2
3  on:
4    push:
5      branches: [ "main" ]
6    pull_request:
7      branches: [ "main" ]
8
9  jobs:
10   build:
11     runs-on: ubuntu-latest
12
13     steps:
14       - uses: actions/checkout@v4
15       - name: Log in to DockerHub
16         run: echo ${ secrets.DOCKER_PASSWORD }} | docker login -u ${ secrets.DOCKER_USERNAME }} --password-stdin
17       - name: Build and Push Docker image
18         run: |
19           IMAGE_TAG=${ github.sha }}
20           docker build -t ${ secrets.DOCKER_USERNAME }}/cicd-pipeline:latest -t ${ secrets.DOCKER_USERNAME }}/cicd-pipeline:$IMAGE_TAG .
21           docker push ${ secrets.DOCKER_USERNAME }}/cicd-pipeline:latest
22           docker push ${ secrets.DOCKER_USERNAME }}/cicd-pipeline:$IMAGE_TAG

```

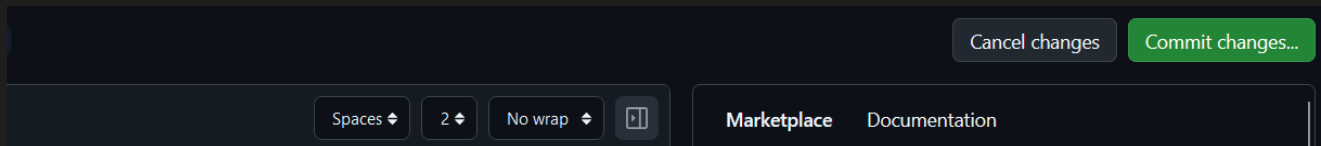
🔗 Version Control!

Aqui criamos duas imagens, uma com a tag "latest", que substituirá a existente, caso exista, e outra com o SHA gerado pelo GitHub, assim, podemos manter e rastrear imagens anteriores.

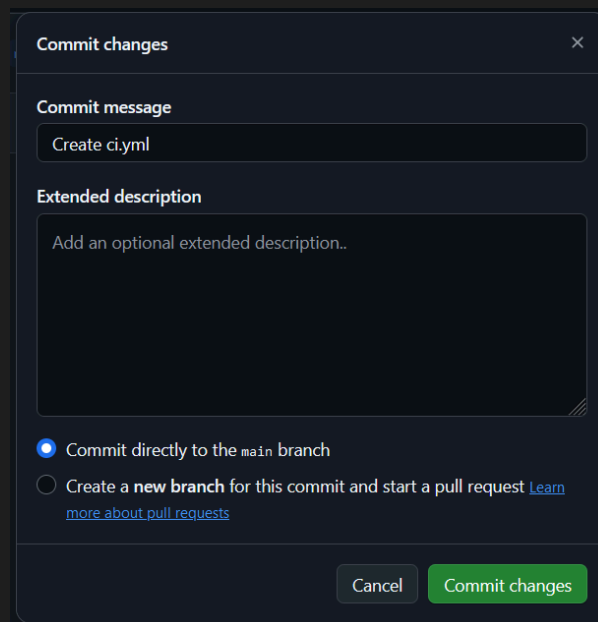
📄 Nome do container

Aqui foi utilizado "cicd-pipeline" como nome, caso tenha escolhido outro nome, sempre que tiver "cicd-pipeline" substituir pelo nome do seu container

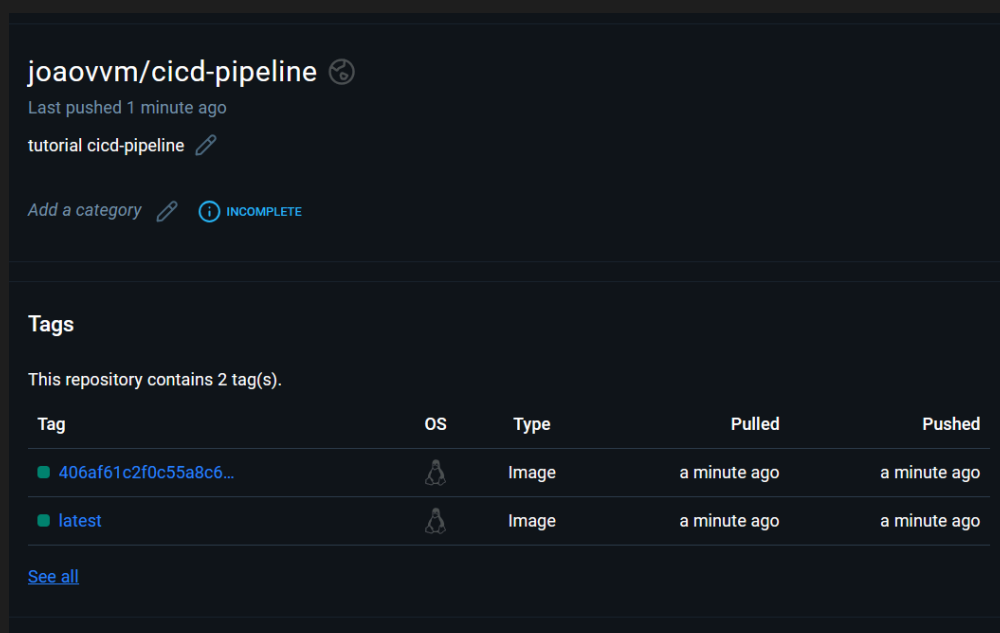
- No canto superior direito clique em "Commit changes"



- Termine o commit.



- O passo anterior irá acionar a Action
- Se tudo estiver correto, a imagem estará disponível no DockerHub!



3.10. Action CD

- Sincronizar o repositório local com o remoto:

```
git pull
```

```
PS C:\Users\joaov\go-app> git pull
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 1.30 KiB | 47.00 KiB/s, done.
From https://github.com/joaobgoode/cicd-pipeline
   395c114..406af61  main      → origin/main
Updating 395c114..406af61
Fast-forward
 .github/workflows/ci.yml | 22 +++++++++++++++++++++
 1 file changed, 22 insertions(+)
 create mode 100644 .github/workflows/ci.yml
```

- No próprio editor, criar um arquivo "*cd.yml*" na pasta `.github/workflows` com o conteúdo:

```
name: Docker Image CD

on:
  workflow_run:
    workflows: ["Docker Image CI"]
    types:
      - completed

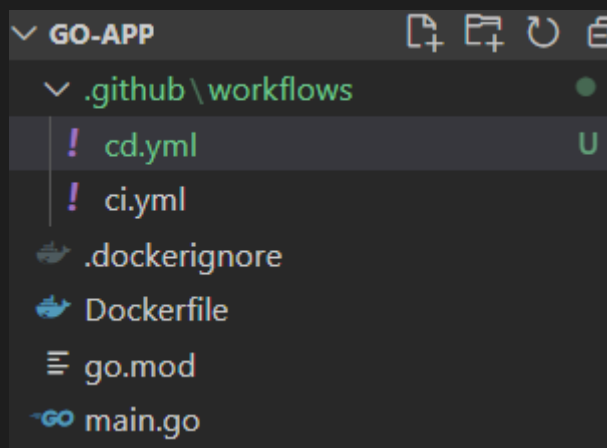
jobs:
  build:

    runs-on: self-hosted

    steps:
      - name: Pull Docker Image from DockerHub
        env:
          DOCKER_USERNAME: ${ secrets.DOCKER_USERNAME }
        run: sudo docker pull $DOCKER_USERNAME/cicd-pipeline:latest

      - name: Delete old Docker Container
        run: sudo docker rm -f cicd-pipeline-container || true

      - name: Run Docker Container
        run: sudo docker run -d -p 8080:8080 --name cicd-pipeline-container
            ${ secrets.DOCKER_USERNAME }/cicd-pipeline
```



- Subir a Action

```
git add .
git commit -m "CD action"
git push
```

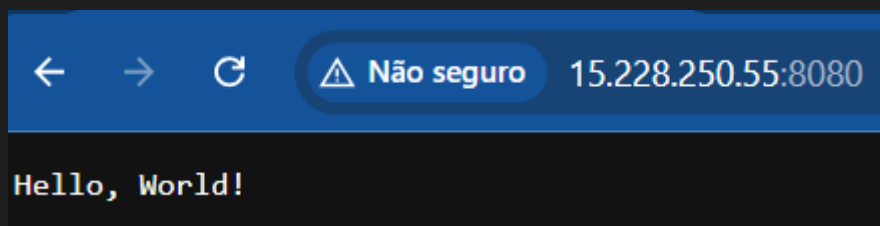
```
PS C:\Users\joaov\go-app> git add .
PS C:\Users\joaov\go-app> git commit -m "CD Action"
[main 0eb60b6] CD Action
 1 file changed, 25 insertions(+)
 create mode 100644 .github/workflows/cd.yml
PS C:\Users\joaov\go-app> git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 657 bytes | 657.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/joaobgoode/cicd-pipeline.git
 406af61..0eb60b6  main -> main
PS C:\Users\joaov\go-app>
```

- O push do passo anterior irá acionar as Actions

🔗 Outra opção!

É possível fazer o *cd.yml* indo em "Actions" > "New Workflow" e seguindo da mesma forma que foi feito o *ci.yml*, mas foi escolhido fazer assim neste tutorial para demonstrar outras formas de se realizar isso!

- Caso tudo esteja de acordo, o deploy já será feito
- Teste no navegador ou com curl!



```
curl "http://IP:8080/"
```

```
PS C:\Users\joaov\go-app> curl "http://15.228.250.55:8080/"  
Hello, World!
```

4. Conclusão

Criar uma pipeline para CI/CD pode parecer complexo no início, especialmente para quem está se aventurando pela primeira vez nesse processo. A configuração inicial exige atenção aos detalhes e uma boa compreensão dos conceitos envolvidos, mas a prática traz familiaridade e confiança. Com o tempo, os desafios são superados, e os benefícios tornam-se evidentes: automação de tarefas repetitivas, maior eficiência no ciclo de desenvolvimento, detecção precoce de erros e maior confiança nas entregas.

Além disso, usar ferramentas como GitHub Actions simplifica significativamente a implementação de pipelines. Sua flexibilidade permite criar processos que independem da linguagem de programação ou do ambiente de deploy da aplicação. Isso torna as pipelines adaptáveis a uma variedade de cenários e garante que a equipe possa focar no que realmente importa: desenvolver soluções de qualidade de forma ágil e consistente. No final, o esforço inicial para criar uma pipeline se paga com um fluxo de trabalho mais robusto e confiável.